

Caching

1 De cache structuur

1. Een 64KiB, direct mapped cache heeft 16 byte blocks. Je mag veronderstellen dat het geheugen byte geadresseerd is (dus 1 woord = 1 byte). Als adressen 32 bits lang zijn, hoeveel bits zijn er dan voorzien voor de tag, de index, en de offset in deze cache?
2. Pas je antwoord aan indien de cache 4-way set associatief is in plaats van direct.
3. Hoeveel bits hebben we voor de index nodig indien de cache volledig associatief is? Verklaar je antwoord.

2 Hit/miss in een 2-way LRU cache

Een 8 byte, 2-way set associatieve cache (met LRU replacement) met 2 byte blocks ontvangt requests voor de volgende (binaire) adressen:

0110, 0000, 0010, 0001, 0011, 0100, 1001, 0000, 1010, 1111, 0111.

Geef na elke access de inhoud van de cache door de bijgevoegde tabel in te vullen, voor elke set moet je de aanwezige tags weergeven.

Adres	Set 0	Set 1	Hit/Miss
0110	leeg/leeg	01/leeg	Hit/Miss
0000	/	/	Hit/Miss
0010	/	/	Hit/Miss
0001	/	/	Hit/Miss
0011	/	/	Hit/Miss
0100	/	/	Hit/Miss
1001	/	/	Hit/Miss
0000	/	/	Hit/Miss
1010	/	/	Hit/Miss
1111	/	/	Hit/Miss
0111	/	/	Hit/Miss

3 Cache hit rate

3M beslist Post-Its te maken door gele vierkanten op wit papier te printen, waarbij voor elk punt in het vierkant de CMYK (cyan, magenta, yellow, black) waarde moet worden ingevuld. 3M neemt je in dienst om drie implementaties te bekijken op een machine met een 2048-byte direct mapped data cache met 32 byte blokken/lijnen. Je krijgt volgende definities mee:

```
struct point_color {
    int c;
    int m;
```

```

    int y;
    int k;
};
struct point_color square[16][16];
register int i, j;

```

Veronderstel verder dat

- $\text{sizeof}(\text{int}) = 4$,
- square begint op memory address 0,
- de cache is initieel leeg,
- de enige memory accesses zijn naar de entries van de array square. De variabelen i en j zijn opgeslagen in de registers.
- het geheugen 4-byte geadresseerd wordt, dus $1 \text{ int} = 1 \text{ woord}$.

Wat is het percentage van de writes in de volgende code die een hit veroorzaken?

Methode 1:

```

for (i=0; i<16; i++) {
    for (j=0; j<16; j++) {
        square[i][j].c = 0;
        square[i][j].m = 0;
        square[i][j].y = 1;
        square[i][j].k = 0;
    }
}

```

Methode 2:

```

for (i=0; i<16; i++) {
    for (j=0; j<16; j++) {
        square[j][i].c = 0;
        square[j][i].m = 0;
        square[j][i].y = 1;
        square[j][i].k = 0;
    }
}

```

Methode 3:

```

for (i=0; i<16; i++) {
    for (j=0; j<16; j++) {
        square[j][i].y = 1;
    }
}

```

```

for (i=0; i<16; i++){
    for (j=0; j<16; j++) {
        square[j][i].c = 0;
        square[j][i].m = 0;
        square[j][i].k = 0;
    }
}

```

4 Cache replacement algoritmen

Ga voor het tree-based en MRU-based pseudo LRU algoritme volgende twee zaken na en bewijs, indien nodig, deze eigenschappen:

1. Stel dat w_i het i -de meest recent gebruikte woord is, voor $i = 1$ tot de set size k . Bij LRU kan enkel woord w_k vervangen worden. Welke woorden kunnen er allemaal vervangen worden indien we een pseudo LRU hanteren?
2. Stel dat een bepaald blok B (d.i., een bepaalde lijn) niet meer gebruikt wordt vanaf een zeker ogenblik. Hoe lang kan blok B dan nog aanwezig blijven in de cache? Anders gezegd: hoeveel replacements kan B maximaal 'overleven'?

5 Access times en hit rates

Laat T_1 de access tijd voorstellen indien een woord zich in de $L1$ cache bevindt. Indien het woord enkel in de $L2$ cache aanwezig is en niet in de $L1$ cache, dan is er een tijd T_2 nodig om het woord over te brengen naar de $L1$ cache. Tenslotte, is er een tijd T_M nodig om een woord van het main geheugen naar de $L2$ cache te verplaatsen. Stel dat $T_1 = 20$ nsec, $T_2 = 100$ nsec en $T_M = 400$ nsec.

1. Indien we respectievelijk een hit rate van .9 en .5 hebben in de $L1$ en $L2$ cache, wat is dan de gemiddelde access time?
2. Indien de hit rate in de $L2$ cache .6 is, wat is dan de hit rate in de $L1$ cache indien de gemiddelde access time gelijk is aan 33 nsec?

6 Direct mapping versus set-associative mapping met random replacement

Beschouw het volgende stuk code uit de theorieslides.

```
float a[1024], b[1024];
for (i=0; i<1024; i++)
    sum += a[i] * b[i];
```

Veronderstel eveneens:

- Cache size: 4 KByte, Block size: 32 byte, Word size: 4 byte
- Float: 4 byte
- b zit vlak na a in het geheugen
- i en sum zijn registers

1. Wat is de hit rate als we direct-mapping veronderstellen ?
2. Als we een 4-way set-associatief geheugen met random replacement veronderstellen, bereken de (benaderde) hit rate.
3. Pas je antwoord op vorige 2 vragen aan, als je weet dat beide arrays elk slechts 256 elementen bevatten. Doe dit eveneens voor 1536.

7 Benchmarking

Teken een grafiek die een cache benchmark, zoals beschreven op p31 van de theoriecursus, illustreert. We beschouwen een systeem dat enkel een $L1$ cache heeft; eventuele TLB en paging effecten hoeven niet weergegeven te worden. De grootte van de $L1$ cache is 32 KByte, een cache lijn bevat 64 bytes en er wordt 8-way associativiteit verondersteld. Een hit heeft een access tijd van 50 ns, een miss kost 500 ns extra. Maak de grafiek, met op de x-as de stride-grootte (logaritmische schaal) en de access tijd op de y-as, voor arraygroottes van 4 tot 512 Kbyte.

Geheugen Management

1 Buddy systeem

1. Beschouw een buddy systeem van 1Mbyte groot. Geef grafisch de toestand van het geheugen weer na volgende acties, als je weet dat het geheugen initieel leeg is.
 - (a) Request A = 70 Kbyte
 - (b) Request B = 35 Kbyte
 - (c) Request C = 80 Kbyte
 - (d) Return A
 - (e) Request D = 60 Kbyte
 - (f) Return B
 - (g) Return D
2. In het standaard buddy systeem krijgen de processen uiteindelijk geheugenblokken toegewezen van machten van 2. Stel dat we in plaats daarvan een Fibonacci reeks willen gebruiken.
 - Leg de werking van zo een systeem uit aan de hand van een kort voorbeeld. (tip: veronderstel dat de totale geheugengrootte een (groot) Fibonacci getal is)
 - Presteert dit systeem beter of slechter dan het standaard buddy systeem qua interne fragmentatie ? Waarom (niet) ?

Paging

1 Omvang van de page tables van een proces

Stel dat we een 32-bit machine hebben met virtuele adressen die zijn opgesplitst in 4 delen van 10, 8, 6 en 8 bits respectievelijk. We hanteren deze adressen in een 3 level page table systeem, waarbij de eerste 10 bits gebruikt worden voor level 1, de volgende 8 bits voor level 2, etc. Een enkele page table entry is 4 bytes groot. Het systeem is byte geadresseerd.

1. Wat is de page size in dit systeem?
2. Stel dat een process 256K geheugen vereist (dat start vanaf adres 0), hoeveel plaats nemen de page tables van dit proces dan in beslag?
3. Stel dat een proces een code segment heeft van 48K vanaf (virtueel) adres 0x10000000, een 600K data segment vanaf adres 0x80000000 en een stack segment van 64K vanaf adres 0xF0000000. Hoeveel plaats vereist de page table van dit proces?

2 Bepaal het aantal levels in een multilevel paging systeem

Stel dat een machine virtuele adressen heeft bestaande uit 36 bits en dat de pages 8K groot zijn. Een enkele page entry is 8 bytes lang.

1. Uit hoeveel paginas bestaat de virtuele adresruimte?
2. Stel dat een gemiddeld proces 8 Gbyte groot is. Hoeveel levels zijn er dan het meest geschikt indien we met een multilevel page table werken (bekijk 1 tot 3 levels)? Weeg de voor- en nadelen tegen elkaar af. Voor het systeem met 2 levels kies je de omvang van een level 2 page table zodanig dat deze precies past in een enkele page. Voor drie levels neem je een min of meer gelijke opdeling voor elk van de velden.

3 Tel het aantal TLB misses

Beschouw volgend stukje code. Stel dat de binaire code op één page past en dat de stack eveneens op een enkele page past. Je mag er bovendien van uit gaan dat bij het aanroepen van de functie, de stack en code page reeds in de TLB cache zitten. Veronderstel verder dat een integer 4 bytes lang is en dat een page 4096 bytes bevat.

```
int a[1024][1024], b[1024][1024], c[1024][1024];
multiply() {
    unsigned i,j,k;
    for(i=0; i< 1024; i++)
        for(j=0; j< 1024; j++)
            for(k=0; k< 1024; k++)
                c[i][j]+=a[i][k]*b[k][j];
}
```

Gegeven dat de TLB cache slechts 8 entries bevat en dat hij een LRU replacement policy gebruikt, wat is dan het aantal TLB misses bij de uitvoering van deze code?

4 Demand paging: goed of slecht

Welke van onderstaande programmeer technieken werkt *goed* in combinatie met een demand paged systeem.

1. Stack
2. Hash table
3. Sequentiële zoekmethode
4. Binaire zoekmethode
5. Pure code
6. Vector operaties (bijvoorbeeld vectorvermenigvuldiging)
7. Indirecte adressen

5 Nuttige ingrepen voor demand paging

Stel dat we enkele statistieken nemen op een demand paged systeem. We merken dat de CPU 20% gebruikt wordt, de disk met de pages 97.3% en de overige I/O devices slechts 5%. Welke ingrepen lijken je doeltreffend om het systeem te verbeteren?

1. Installeer een snellere CPU
2. Installeer een grotere paging disk
3. Verhoog de mate van multiprogrammeren
4. Verlaag de mate van multiprogrammeren
5. Installeer meer geheugen
6. Installeer een snellere hard disk
7. Laat prepaging toe
8. Verhoog de page size

6 Inverted versus standaard page tables

Indien virtuele adressen 64 bits lang zijn, de page size 1Kbyte is, de omvang van het fysieke geheugen is 64Mbyte en een PTE neemt twee bytes in beslag, wat is dan de totale grootte van de inverted en standaard page table (indien elke byte geadresseerd wordt)? Maak hiervoor eerst een schatting van het aantal bytes een inverted page table entry nodig heeft, door na te gaan welke velden er minstens in moeten zitten, en hoeveel bits ze elk moeten bevatten.

7 Inverted page table

Beschouw een systeem met 16KB geheugen, pages van 1KB groot, een inverted page table en een full-associative TLB met 2 entries en een LRU replacement policy. De gebruikte hash functie is $f(PID, VPN) = (10 * PID + VPN) \bmod 24$. Het systeem heeft 0,5ns nodig om de TLB uit te lezen, 100ns om het main memory uit te lezen en 10ms om de harde schijf raad te plegen. Vul de hieronder gegeven tabel met page requests verder aan gebruik makend van de gegeven hash anchor table en de inverted page table (IPT). In de IPT wordt “T” gebruikt als symbool om aan te duiden dat er geen volgende page table entry (PTE) is.

In de TLB kolom geef je de inhoud van de TLB weer nadat de vertaling gebeurd is. Maak hiervoor gebruik van de notatie “(VPN, Frame number)” voor elke lijn in de cache, gebruik “-” als de desbetreffende lijn leeg is. In de kolommen genaamd “Frame number”, “TLB hits” en “Page fault” geef je respectievelijk de physical frame number weer, of er tijdens de vertaling een TLB hit was en of het aanleiding gaf tot een page fault of niet. Ten slotte vul je in de laatste kolom de tijd in die nodig was om de vertaling uit te voeren. We veronderstellen dat de buffer in parallel geupdate kan worden na het vertalen en het uitvoeren van de volgende instructie, hiervoor hoeft je dus geen tijd te rekenen. In het geval van eventuele page faults hoeft je de datastructuren niet aan te passen, wel dien je de tijd te geven die nodig was om erachter te komen dat er een page fault is.

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	7	3	12	15	13	6	2	4	0	14	8	5	15	11	10	8	5	7	9	10	12	6	13	1

Tabel 1: Hash anchor table

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
PID	0	3	2	3	0	8	0	2	2	0	8	6	0	2	2	4
VPN	8	17	8	4	1	3	5	4	2	3	13	1	2	32	0	4
Next PTE	2	T	T	4	T	T	10	12	3	T	0	9	14	8	T	11

Tabel 2: Inverted Page table

PID	VPN	TLB	Frame number	TLB hits	Page fault	Latency (ns)
0	1			Hit/Miss	Yes/No	
0	2			Hit/Miss	Yes/No	
0	1			Hit/Miss	Yes/No	
0	3			Hit/Miss	Yes/No	
8	3			Hit/Miss	Yes/No	
0	3			Hit/Miss	Yes/No	
2	2			Hit/Miss	Yes/No	
2	1			Hit/Miss	Yes/No	
8	13			Hit/Miss	Yes/No	
8	3			Hit/Miss	Yes/No	

8 Tel het aantal page faults

Beschouw volgende sequentie van paginas:

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.

Hoeveel page faults treden er op indien dit process over 1, 2, ..., 6 of 7 frames kan beschikken. Ga dit na voor volgende drie replacement algoritmen: LRU, FIFO en het MIN algoritme. Initieel zijn er geen paginas in het geheugen aanwezig.

9 Segmented FIFO

Waar of vals?

1. Voor alle pages in L1 geldt dat ze recenter opgevraagd zijn dan deze in L2.

2. Pages in L1 zitten in LRU volgorde.
3. Pages in L2 zitten in LRU volgorde.
4. $|L2| - 1$ meest recent gebruikte pages zijn safe (kunnen niet vervangen worden).
5. $|L2|$ meest recent gebruikte pages zijn safe.
6. Een segmented FIFO systeem met $|L1| = |L2| = 2$ is equivalent aan een tree-based pseudo-LRU systeem met 4 pages.

10 Benchmarking

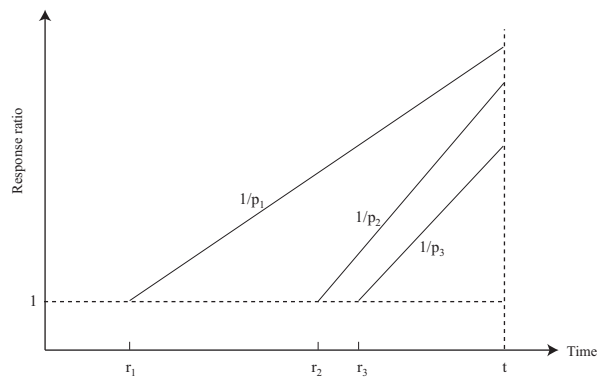
Teken een grafiek die een cache benchmark, zoals beschreven op p31 van de theorie cursus, illustreert. We beschouwen een systeem dat een L1 cache en een TLB cache heeft. De grootte van de L1 cache omvat 256 KByte, een cache lijn bevat 32 bytes en er wordt direct-mapping verondersteld. Een L1 cache access neemt 250ns in beslag, een L1 cache miss zorgt voor 500ns extra, een TLB miss nog eens 1000ns. De TLB cache is 4-way associatief, bevat 128 entries, en de page size is 8192 bytes.

Maak de grafiek, met op de x-as de stride-grootte (logaritmische schaal) en de access tijd op de y-as, voor arraygroottes van 4 Kbyte tot 4096 Kbyte.

Uniprocessor scheduling

1 Een variant op highest response ratio next

Een variant van HRRN, minimax response ratio genaamd, heeft als doelstelling de maximale response ratio te minimaliseren. Bedenk een scheduling policy die hieraan voldoet. Inspireer je op het voorbeeld van Figuur 1; stel dat we een scheduling beslissing moeten maken op tijd t . In welke volgorde moeten taken t_1 , t_2 en t_3 worden uitgevoerd (vanaf tijd t), opdat de maximale response ratio $\max_s((w + p_s)/p_s)$ zo klein mogelijk is?



Figuur 1: Reponse ratio in functie van de tijd

2 Rate monotonic scheduling

Zoals reeds vermeld in de cursus, kan men aantonen dat RMS steeds de deadlines zal respecteren indien $U = \sum_{j=1}^n p_j/T_j \leq n(2^{1/n} - 1)$

1. Geef een voorbeeld van jobs (minstens 2 jobs) die met RMS hun deadlines halen, en U gelijk is aan 1.
2. Geef een voorbeeld van jobs die met RMS hun deadlines NIET halen, en U is hoogstens 0.95.
3. Geef een voorbeeld van jobs die met RMS hun deadlines NIET halen, en U is hoogstens 0.84.

Disk Management

1 Random seek

We beschouwen een harddisk met N tracks. Veronderstel nu dat de gevraagde sectoren random verdeeld zijn over alle tracks en onafhankelijk van elkaar. Bereken nu het gemiddeld aantal tracks dat de diskhead moet afleggen tussen twee requests, als volgt:

1. Wat is de kans dat de disk head op positie i staat en naar positie j moet?
2. Hoeveel tracks moeten er afgelegd worden van i naar j ?
3. Combineer 1 en 2 door alle mogelijke combinaties van i en j na te gaan, en werk uit.
4. Gegeven dat N groot is, geef een benadering voor het aantal af te leggen tracks, relatief t.o.v. het totaal aantal tracks.

2 Scheduling algoritmes

Veronderstel dat we een IO controller hebben die het SSTF algoritme hanteert, en dat de diskhead op de middenste track van de disk staat.

1. Geef nu een reeks van 8 requests op, waarbij de disk head tussen elke request over het midden van de disk moet langskomen. Geef de minimale (in de zin van totaal af te leggen tracks) oplossing voor dit probleem. (Je mag de requests noteren relatief ten opzichte van de middenste track; met andere woorden als de middenste track 512 is, mag je track 511 aangeven als -1 .)
2. Berekenen het totaal aantal trackwissels voor dit request patroon.
3. Laat hetzelfde patroon afhandelen door een LOOK algoritme, en bereken hiervoor het aantal trackwissels. Vergelijk.

3 RAID efficiënt?

Gegeven dat je 12 harddisks hebt van elk 40GB, ga na voor volgende RAID configuraties, hoeveel GB je uiteindelijk op dit RAID systeem kan plaatsen?

- RAID 0+1
- RAID 1+0
- RAID 0+5 (alle 3 mogelijke opsplitsingen)
- RAID 5+0 (alle 3 mogelijke opsplitsingen)

4 RAID: robuust?

Gegeven dat je 12 harddisks hebt, ga na voor volgende RAID configuraties, hoeveel disks er maximaal mogen uitvallen, zonder dat het systeem faalt (niet noodzakelijk onherstelbaar). Berekenen hierna de kans dat het systeem operationeel blijft, wanneer twee hard disks falen.

- RAID 0+1
- RAID 1+0
- RAID 0+5 (alle 3 mogelijke opsplitsingen)
- RAID 5+0 (alle 3 mogelijke opsplitsingen)

Concurrency

1 Producer/consumer probleem

Geef een mogelijke implementatie voor het producer/consumer probleem met semaforen, in de veronderstelling dat de buffer waarin de data wordt weggeschreven nu eindig is; met andere woorden, als de producer een item wilt wegschrijven en de buffer is vol, moet deze wachten tot er terug een plek vrij is.

2 Concurrency

Beschouw het volgende stuk code:

```
const int n = 50;
int tally;
void total() {
    int count;
    for (count = 1; count <= n; count++) {
        tally=tally+1;
    }
}

void main() {
    tally = 0;
    parbegin(total(),total()); // parallel uitvoeren
    cout << tally << endl;
}
```

1. Wat is de minimale en maximale waarde die de variabele *tally* kan aannemen, bij het einde van het programma?
2. Stel dat er N parallele instanties van *total* worden uitgevoerd, wat verandert er aan deze grenzen?

3 Paper review

Papers reviewen behoort tot de talrijke activiteiten van onderzoekers. Het doel hiervan bestaat erin om een paper “na te kijken”, onder andere op correctheid, alvorens deze kan gepubliceerd worden in een tijdschrift, of voorgesteld worden op een conferentie. Onderstaande *comment* is in 1965 ingestuurd naar het vrij hoog aangeschreven tijdschrift Communications of the ACM. Review deze comment, in Figuur 2, op correctheid. (Tip: zoek een uitvoervolgorde, waarbij twee processen tegelijkertijd toegang verkrijgen tot de kritische sectie)

✓ Comments on a Problem in Concurrent Programming Control

Dear Editor:

I would like to comment on Mr. Dijkstra's solution [Solution of a problem in concurrent programming control. *Comm ACM* 8 (Sept. 1965), 569] to a messy problem that is hardly academic. We are using it now on a multiple computer complex.

When there are only two computers, the algorithm may be simplified to the following:

```
Boolean array  $b(0; 1)$  integer  $k, i, j$ ,  
comment This is the program for computer  $i$ , which may be  
either 0 or 1, computer  $j \neq i$  is the other one, 1 or 0;  
 $C0$ :  $b(i) := \text{false}$ ;  
 $C1$ : if  $k \neq i$  then begin  
 $C2$ : if not  $b(j)$  then go to  $C2$ ;  
    else  $k := i$ ; go to  $C1$  end;  
    else critical section;  
     $b(i) := \text{true}$ ;  
    remainder of program;  
    go to  $C0$ ;  
    end
```

Mr. Dijkstra has come up with a clever solution to a really practical problem.

HARRIS HYMAN
Munitype
New York, New York

Figure 2: Comment submitted to Communications of the ACM.

4 Binaire semaforen

```
void semWait(semaphore s) {
    semWaitB(mutex);
    s--;
    if (s < 0) {
        semSignalB(mutex);
        semWaitB(delay);
    } else
        semSignalB(mutex);
}

void semSignal(semaphore s) {
    semWaitB(mutex);
    s++;
    if (s <= 0)
        semSignalB(delay);
    semSignalB(mutex);
}
```

Deze code zou een mogelijke implementatie kunnen voorstellen, die *counting semaphores* realiseert, aan de hand van binaire semaforen. Helaas kan deze code nog een probleem opleveren. Zoek dit probleem, en pas de code aan zodat je een correcte oplossing verkrijgt.

5 Mutual exclusion

Gegeven de onderstaande code, beantwoord de volgende vragen.

- Is mutual exclusion gegarandeerd voor de kritische sectie? Leg uit.
- Is het mogelijk om in een deadlock terecht te komen? Leg uit.
- Is starvation mogelijk? Leg uit.

```
1  globale variabelen:
2  int turn = -1;
3  bool busy = false;
4
5  thread_i:
6      while(turn != i) {
7          while(busy) {
8              turn = i;
9          }
10         busy = true;
11     }
12 }
13
14 <critical section>
15
16     busy = false;
```

6 Model van een pretpark

Beschouw een attractie in een pretpark bestaande uit een rondrit in een 1-persoonswagentje door een spookhuis. In totaal beschikt de attractie over n wagentjes op een vast spoor (de wagentjes kunnen elkaar niet inhalen) en om energie te besparen, rijden enkel de volle wagentjes rond; de overigen staan in het station te wachten op passagiers. De m bezoekers van het pretpark lopen een tijd rond in het pretpark, komen aan bij het spookhuis, wachten eventueel op een wagentje in een FIFO queue, stappen in het eerst beschikbare wagentje in, genieten van de attractie, stappen uit en vervolledigen dan hun bezoek aan het park.

Als we de bezoekers en de wagentjes beide modelleren als processen die zich moeten synchroniseren (gebruik makend van semaforen), moet er zeker voorkomen worden dat

- Een wagentje kan vertrekken terwijl het nog niet vol is.
- Een wagentje kan vertrekken terwijl een passagier in of uit dat wagentje stapt.
- Passagiers te vroeg uitstappen, i.e. ze mogen het wagentje niet verlaten tijdens de rit.

Gegeven de globale variabelen:

```
cars_avail = semaphore initialized on 0;
passenger_safe[] = array of n semaphores initialized on 0;
ride_done[] = array of n semaphores initialized on 0;
```

Volledige code voor de wagentjes:

```
car_j:
while (turned_on) {
    signal(cars_avail)
    <pick up human i>
    wait(passenger_safe[j])
    <drive around with human i>
    signal(ride_done[j])
    <drop off human i>
    wait(passengers_safe[j])
}
```

Gedeeltelijke code voor de passagiers:

```
human_i:
...
<wander around in park>
...
<get into car j>
...
<get scared>
...
<get out of car j>
...
<complete visit to park>
...
```

- Vervolledig de code van de bezoekers. Maak hierbij enkel gebruik van de gekende semafooroperaties.
- Wat moet er veranderd worden indien de wagentjes k personen kunnen vervoeren? Geef de aangepaste code voor zowel de wagentjes als de bezoekers. We gaan er hierbij van uit dat passagiers onderling niet communiceren, en niet kunnen achterhalen hoeveel passagiers er reeds in het wagentje aanwezig zijn. (2pt)
- Zijn er randvoorwaarden op k en m ? Leg uit. (1pt)

Cache coherency

1 MESI protocol

Een systeem heeft twee processoren met elk een volledig associatieve cache van 4 lijnen. Beide caches gebruiken het tree-based replacement algoritme, zijn initieel leeg en de processoren gebruiken het MESI protocol. Vervolledig volgende tabel. Bij de requests staat *R* voor *read* en *W* voor *write*.

Memory access		Cache 1	Cache 2	Opmerkingen
Proc 1	Proc 2	- - - - / I I I I	- - - - / I I I I	
R1		1 - - - / E I I I	- - - - / I I I I	
R2		1 - 2 - / E I E I	- - - - / I I I I	
	W3			
	R4			
W5				
R6				
	R2			
W2				
	W1			
	R3			
R1				
	R6			
W7				
	R1			
	W7			
R1				
R6				

Multiprocessor scheduling

1 List Scheduling Algoritme

Geef een (algemeen) voorbeeld waarbij LS ook effectief $2 - 1/m$ keer slechter presteert dan een optimaal algoritme, in functie van de maximale completion time; met andere woorden toon aan dat deze bovengrens niet verbeterd kan worden. Probeer eerst een voorbeeld te vinden voor $m = 2, 3$, geef daarna een algemeen voorbeeld.

2 Longest Processing Time First

Geef een (algemeen) voorbeeld waarbij LPT ook effectief $4/3 - 1/3m$ keer slechter presteert dan een optimaal algoritme, in functie van de maximale completion time; met andere woorden toon aan dat deze bovengrens niet verbeterd kan worden. Probeer eerst een voorbeeld te vinden voor $m = 2, 3, 4$, geef daarna een algemeen voorbeeld.